# CS7643: Deep Learning
# Assignment 1

Instructor: Zsolt Kira

Fall 2026

- This assignment is due on the date/time posted on Canvas. We will have a 48-hour grace period for this assignment. However, no questions about the assignment are answered during the grace period in any way.
- Discussion is encouraged, but each student must write his/her own answers and explicitly mention any collaborators.
- Each student is expected to respect and follow the GT Honor Code. **We will apply anti-cheating software to check for plagiarism**. Anyone who is flagged by the software will automatically receive 0 for homework and be reported to OSI.
- Please **do not change the filenames and function definitions** in the skeleton code provided, as this will cause the test scripts to fail and you will not receive points in those failed tests. You may also **NOT** change the import modules in each file or import additional modules.
- It is your responsibility to ensure that all code and other deliverables are in the correct format and that your submission compiles and runs. We will not manually check your code (this is not feasible given the class size). Thus, **non-runnable code in our test environment will directly lead to a score of 0**. Also, your entire programming parts will **NOT** be graded and given a 0 score if your code prints out anything that is not asked in each question.

# Theory Problem Set

1. **The Softmax-CE Computation Graph**

   **Problem Context:** In modern deep learning, we treat the Softmax activation and the Cross-Entropy (CE) loss as a single "fused" block. This question guides you through the derivation of the gradient for this block—a result you will use directly in your implementation of `softmax_regression.py` and `two_layer_nn.py`.

   Consider a simple computation graph with a 3-length vector input $\vec{z}$:

   $$\vec{z} = \begin{bmatrix} z_1 \\ z_2 \\ z_3 \end{bmatrix} \rightarrow \Big(\text{SM}\Big) \rightarrow \vec{p} = \begin{bmatrix} p_1 \\ p_2 \\ p_3 \end{bmatrix} \rightarrow \Big(\text{CE}\Big) \rightarrow L$$

   (a) **Derive the Softmax Gradient**

   The softmax output for a single component in the vector $\vec{p}$ is:

   $$p_i = \frac{e^{z_i}}{\sum_k e^{z_k}}$$

   Derive the Jacobian matrix $\dfrac{\partial \vec{p}}{\partial \vec{z}}$. Specifically, show that

   $$\frac{\partial p_i}{\partial z_j} = p_i(\delta_{ij} - p_j)$$

   where $\delta_{ij}$ is the Kronecker delta (1 if $i = j$, 0 otherwise).

   *Note: Please use numerator layout for the Jacobian.*

   (b) **Derive the Cross-Entropy Gradient**

   The Cross-Entropy loss is:

   $$L = -\sum_k y_k \ln(p_k)$$

   where $y$ can be represented as a one-hot encoding of the ground truth label. Derive $\dfrac{\partial L}{\partial \vec{p}}$.

   *Hint: Remember that this is the derivative of a scalar with respect to a vector.*

   (c) **Put it All Together (The Matrix-Vector Product)**

   Apply the chain rule to find:

   $$\frac{\partial L}{\partial \vec{z}} = \left(\frac{\partial \vec{p}}{\partial \vec{z}}\right)^T \cdot \frac{\partial L}{\partial \vec{p}} = \vec{p} - \vec{y}$$

   where $\vec{y}$ is a vector representation of the one-hot encoding of the ground truth.

**Note:** We use column-vector gradients with numerator-layout Jacobians. Under this convention,

$$\frac{\partial L}{\partial \vec{z}} = \left(\frac{\partial \vec{p}}{\partial \vec{z}}\right)^T \frac{\partial L}{\partial \vec{p}}.$$

The Softmax Jacobian $J$ is symmetric, so $J^T = J$ numerically, but we include the transpose for mathematical correctness.

(d) **Extension to Batched Data**

The prior sections covered the derivation of the gradient for a single sample. In your implementation, you deal with a batch of $N$ samples, and `cross_entropy_loss` returns the **mean** loss over those $N$ samples (as in `_base_network.py`).

Explain how the per-example result $\vec{p} - \vec{y}$ extends to a matrix of shape ($N$, num_classes). What is the gradient of the mean batch loss with respect to the pre-softmax scores? How does this simplify your code in the `forward` method when computing gradients?

*Hint: Think of the matrix as a batch of independent vectors, then account for the $1/N$ factor from averaging the loss.*

2. **Linear Threshold Neurons and XOR**

A linear threshold neuron can represent simple Boolean functions such as AND and OR. Formally, given weights $w \in \mathbb{R}^2$, bias $b \in \mathbb{R}$, and inputs $x \in \{0, 1\}^2$, the neuron computes:

$$f(x) = \begin{cases} 1 & \text{if } w^T x + b \geq 0 \\ 0 & \text{if } w^T x + b < 0 \end{cases}$$

Prove that the XOR function cannot be represented by a model of this form. A geometric argument is sufficient.

# Paper Review and Questions

In this section, you must choose **one** of the papers below and complete the following:

1. Provide a short review of the paper.
2. Answer paper-specific questions.

**Review Guidelines**: Please restrict your reviews to no more than 350 words and answers to questions to no more than 350 words per question. The review part (1) should include answers to the following:

- What is the main contribution of this paper? In other words, briefly summarize its key insights. What are some strengths and weaknesses of this paper?
- What is your personal takeaway from this paper? This could be expressed either in terms of your perceived novelty of this paper compared to others you've read in the field, potential future directions of research in the area that the authors haven't addressed, or anything else that struck you as being noteworthy.

## Paper Choice 1:

The first of our paper reviews for this course comes from a much-acclaimed spotlight presentation at NeurIPS 2019 on the topic 'Weight Agnostic Neural Networks' by Adam Gaier and David Ha at Google Brain.

The paper presents a very interesting proposition that, through a series of experiments, re-examines some fundamental notions about neural networks - in particular, the comparative importance of architectures and weights in a network's predictive performance.

The paper can be viewed here. The authors have also written a blog post with intuitive visualizations to help understand its key concepts better.

### Questions for this paper:

1. The traditional view of optimization in deep learning (and often in general) is that we are searching the space of weights to find the best ones. In other words, learning is a *search* problem. How would you view the above paper from the perspective of search?
2. One of the key aspects of deep learning is that given a parameterized function, we can find weights to represent *any* function if it has sufficient depth and complexity. What does this paper say about the representational power of *architectures* given a fixed method for determining weights? Does the method for determining the weights matter? Do you think these two have equal representational power? Why or why not?

## Paper Choice 2:

The second paper is again one that questions conventional wisdom and shows that large neural networks can actually fit *random* labels, that is labels that remain fixed but, for example, have no relationship to what is actually in the image. The paper title is "Understanding deep learning requires rethinking generalization" and can be found here.

**Questions for this paper:**

1. If neural networks can "memorize" the data, which is the only thing they can do for random label assignments that don't correlate with patterns in the data, why do you think neural networks learn more meaningful, generalizable representations when there *are* meaningful patterns in the data?
2. How does this finding align or not align with your understanding of machine learning and generalization?

# Coding: Implement and Train a Network on MNIST

## Overview

Deep Neural Networks are becoming more and more popular and widely applied to many ML-related domains. In this assignment, you will complete a simple pipeline of training neural networks to recognize MNIST Handwritten Digits. You will implement two neural network architectures along with the code to load data, train and optimize these networks. You will also run different experiments on your model to complete a short report. Be sure to use the report template that we give you and fill in your information on the first page.

The notebook `CS7643-Assignment1.ipynb` contains the main logic of this assignment. You can run it either on **Google Colab** or **locally** (e.g., in VS Code or JupyterLab). The environment setup cell at the top of the notebook will automatically detect which platform you are using and configure paths accordingly—no manual changes are needed when running locally.

There are three pre-defined config files under `./configs`. Two of them are default hyperparameters for models that you will implement in the assignment (Softmax Regression and 2-layer MLP). The correctness of your implementation is partially judged by the model performance on these default hyperparameters; therefore, do **NOT** modify values in these config files. The third config file, `config_exp.json`, is used for your hyperparameter tuning experiments (details in Section 5) and you are free to modify values of the hyperparameters in this file.

To run an experiment, execute the experiment cell in the notebook and change the `config_file` variable to select which configuration to use:

```
config_file = "config_exp"
# options: "config_exp", "config_softmax", "config_two_layer_net"
```

The notebook trains a model with the number of epochs specified in the config file. At the end of each epoch, it evaluates the model on the validation set. After training completes, accuracy and loss curves are plotted for both training and validation sets.

## Code Test

There are two ways to test your implementation:

(a) **Notebook Unit Tests:** Public unit tests are provided in the `tests/` directory of the assignment repository. In the notebook, each section includes test cells that run these tests automatically using `pytest`. You can also run all tests at once using the "Run All Tests" cell near the end of the notebook, or from a terminal with:

```
pytest tests/ -v
```

However, passing all local tests neither means your code is free of bugs nor guarantees that you will receive full credits for the coding section. Your code will be graded by the Gradescope Autograder (see below for more details). There will be additional tests on Gradescope which are not present in your local unit tests.

(b) **Gradescope Autograder:** You may also submit your code as specified in Section 6 for testing. The autograder will return the results of public tests, but not the private tests; both of which are used to calculate grades for the coding sections. Note that we do not recommend using Gradescope Autograder as your primary testing method during development because the utility may **NOT** be available at all times.

# 1 Data Loading

Data loading is the very first step of any machine learning pipeline. The notebook handles this automatically—when you run the MNIST loading cell, it will download the dataset on first run and cache it in the `./data` directory for subsequent runs:

```
from cs7643.mnist import MNIST

train_ds = MNIST(os.path.join(PROJECT_ROOT, 'data'), download=True, train=True)
test_ds = MNIST(os.path.join(PROJECT_ROOT, 'data'), download=True, train=False)
```

This downloads `mnist_train.csv` and `mnist_test.csv` (from https://pjreddie.com/media/files/) into `./data/MNIST/raw/`. If the data already exists locally, the download is skipped automatically.

## 1.1 Data Preparation

To avoid the choice of hyperparameters overfitting the training data, it is a common practice to split the training dataset into the actual training data and validation data and perform hyperparameter tuning based on results on validation data. Additionally, in deep learning, training data is often forwarded to models in **batches** for faster training time and noise reduction.

In our pipeline, we first load the entire MNIST data into the system, followed by a training/validation split on the training set. We simply use the **first 80%** of the training set as our training data and use the rest as our validation data. We also want to organize our data (training, validation, and test) in batches and use different combinations of batches in different epochs for training data. Therefore, your tasks are as follows:

(a) Follow the instructions in code to complete `load_mnist_trainval` in `./data/data_processing.py` for training/validation split.

(b) Follow the instructions in code to complete `generate_batched_data` in `./data/data_processing.py` to organize data in batches.

You can test your data loading code by running the corresponding test cells in the notebook, or from a terminal:

```
pytest tests/test_loading.py -v
```

# 2 Model Implementation

You will now implement two networks from scratch: a simple softmax regression and a two-layer multi-layer perceptron (MLP). Definitions of these classes can be found in `./models`.

Weights of each model will be randomly initialized upon construction and stored in a weight dictionary. Meanwhile, a corresponding gradient dictionary is also created and initialized to zeros. Each model only has one public method called `forward`, which takes input of batched data and corresponding labels and returns the loss and accuracy of the batch. Meanwhile, it computes gradients of all weights of the model (even though the method is called forward!) based on the training batch.

## 2.1 Utility Functions

There are a few useful methods defined in `./models/_base_network.py` that can be shared by both models. Your first task is to implement them based on instructions in `_base_network.py`:

(a) **Activation Functions.** There are two activation functions needed for this assignment: ReLU and Sigmoid. Implement both functions as well as their derivatives in `./models/_base_network.py` (i.e., `sigmoid`, `sigmoid_dev`, `ReLU`, and `ReLU_dev`). Test your methods with:

```
pytest tests/test_activation.py -v
```

(b) **Loss Functions.** The loss function used in this assignment is Cross Entropy Loss. You will need to implement both the Softmax function and the computation of Cross Entropy Loss in `./models/_base_network.py`. Test with:

```
pytest tests/test_loss.py -v
```

(c) **Accuracy.** We are also interested in knowing how our model is doing on a given batch of data. Therefore, you may want to implement the `compute_accuracy` method in `./models/_base_network.py` to compute the accuracy of a given batch.

## 2.2 Model Implementation

You will implement the training processes of a simple Softmax Regression and a two-layer MLP in this section. The Softmax Regression is composed of a fully-connected layer followed by a ReLU activation. The two-layer MLP is composed of two fully-connected layers with a Sigmoid activation in between. Note that the Softmax Regression model has no bias terms, while the two-layer MLP model does use biases. Also, don't forget the softmax function before computing your loss!

(a) Implement the `forward` method in `softmax_regression.py` as well as `two_layer_nn.py`. If the `mode` argument is `train`, compute gradients of weights and store the

gradients in the gradient dictionary. Otherwise, simply return the loss and accuracy. Test with:

```
pytest tests/test_network.py -v
```

# 3 Optimizer

We will use an optimizer to update weights of models. An optimizer is initialized with a specific learning rate and a regularization coefficients. Before updating model weights, the optimizer applies L2 regularization on the model:

$$J = L_{CE} + \frac{1}{2}\lambda \sum_{i=1}^{N} w_i^2 \tag{1}$$

where $J$ is the overall loss and $L_{CE}$ is the Cross-Entropy loss computed between predictions and labels.

You will also implement an vanilla SGD optimizer.The update rule is as follows:

$$\theta^{t+1} = \theta^t - \eta \nabla_\theta J(\theta) \tag{2}$$

where $\theta$ is the model parameter, $\eta$ stands for learning rate and the $\nabla$ term corresponds to the gradient of the parameter.

In summary, your tasks are as follows:

(a) Follow instructions in the code and implement `apply_regularization` in `_base_optimizer.py`. Remember, you may **NOT** want to apply regularization on bias terms!

(b) Implement the `update` method in `sgd.py` based on the discussion of the update rule above.

Test your optimizer by running:

```
pytest tests/test_training.py -v
```

# 4 Visualization

It is always a good practice to monitor the training process by observing the learning curves. The notebook automatically plots accuracy and loss curves for both training and validation data at the end of each experiment run. A sample plot of learning curves can be found in Figure 1.

The plotting is handled by the `plot_curve` function in `./cs7643/solver.py`. No implementation is needed for this section—the curves are generated automatically when you run the experiment cell in the notebook.

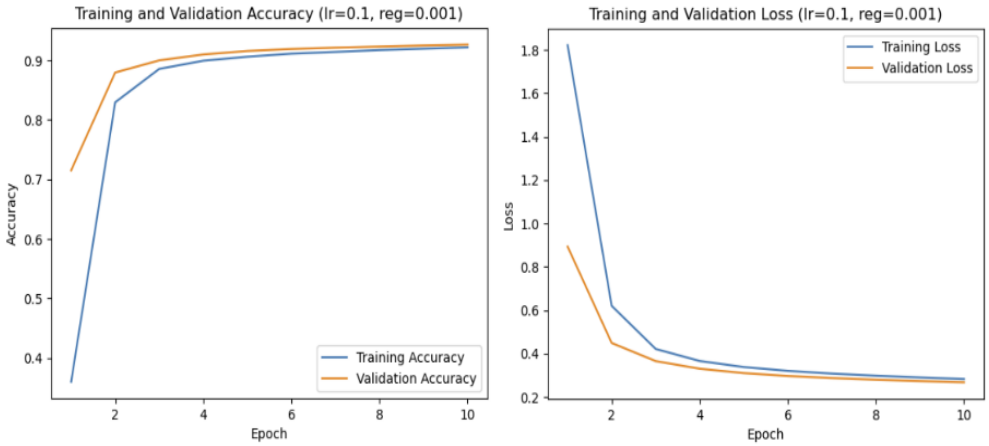



Figure 1: Example plot of learning curves

# 5 Experiments

Now, you have completed the entire training process. It's time to play with your model a little. You will use your implementation of the two-layer MLP for this section. There are different combinations of your hyperparameters specified in the report template and your tasks are to tune those parameters and report your observations by answering questions in the report template. We provide a default config file `config_exp.json` in `./configs`. When tuning a specific hyperparameter (e.g., the learning rate), please leave all other hyperparameters as-is in the default config file.

To run an experiment, execute the experiment cell in the notebook and set the `config_file` variable:

```
config_file = "config_exp"
```

(a) You will try out different values of learning rates and report your observations in the report file.

(b) You will try out different values of regularization coefficients and report your observations in the report file.

(c) You will try your best to tune the hyperparameters for best accuracy.

(d) When tuning for best accuracy, tuning just epochs is not interesting. Tune at least 3 hyperparameters (not including epochs). You may increase or decrease epochs—it does not count as 1 of the 3.

(e) When tuning for best accuracy, the best model should have a marked improvement compared to the default hyperparameters.

(f) When reporting observations be aware of applying good scientific methods.

# 6 Deliverables

## 6.1 Coding

To submit your code to Gradescope, run the final cell in the notebook to generate the submission ZIP file:

```
from cs7643.submit import make_a1_submission
make_a1_submission(PROJECT_ROOT)
```

This creates `assignment_1_submission.zip` in your project directory. Upload this file to Gradescope.

**Important:** Save all `.py` files before running the submission cell.

## 6.2 Writeup

You will also need to submit a report summarizing your experimental results and findings as specified in Section 5. Again, we provide a starting template for you and your task is just to answer each question in the template. For whichever questions asking for plots, please include plots from all your experiments.

Note: Explanations should offer some intuition on *why* certain results might have been observed using your knowledge of Machine Learning. When tuning hyperparameters, explain the reasoning behind the choices. If you need more than one slide for a question, you are free to create new slides **right after the given one**.

You will need to export your report in **pdf** format and submit to Gradescope. You should combine your answers to the theory questions, paper review, and report into one pdf and submit it to the "Assignment 1 Writeup" assignment in Gradescope. **When submitting to Gradescope, make sure you select ALL corresponding slides for each question. Failing to do so will result in -1 point for each incorrectly tagged question, with future assignments having a more severe penalty.**